

PRD — 941

Product Requirements Document

Versión: 1.0

Estado: Documento base para desarrollo

Producto: 941

Categoría: Plataforma mundial de tiempo

Benchmark principal: Time.is

Herramienta de desarrollo: Google Antigravity

1. Resumen del producto

941 será una plataforma web internacional dedicada a proporcionar información precisa, rápida y visualmente refinada sobre el tiempo.

El producto tendrá como núcleo un **reloj mundial de alta precisión**, pero evolucionará hacia un ecosistema de herramientas relacionadas con:

- hora actual
- ciudades
- zonas horarias
- comparación de horarios
- conversión de tiempo
- calendario
- amanecer y atardecer

- countdown
- timer
- Unix timestamp
- planificación de reuniones internacionales

El producto deberá combinar la **simplicidad de [Time.is](#)** con una experiencia visual más moderna y una arquitectura diseñada desde el principio para SEO y escalabilidad.

2. Objetivo principal

Construir una plataforma capaz de responder de forma inmediata a la pregunta:

¿Qué hora es?

y posteriormente permitir al usuario responder preguntas más complejas:

¿Qué hora es en otra ciudad?

¿Cuál es la diferencia horaria?

¿Cuándo sería mejor reunirnos?

¿Qué zona horaria utiliza este lugar?

¿Cuánto falta para determinado evento?

¿Cuál es el Unix timestamp?

Todo desde una experiencia coherente.

3. Objetivos del MVP

La primera versión deberá lograr:

1. Mostrar la hora actual con alta precisión.
2. Detectar o permitir seleccionar una ubicación.
3. Mostrar fecha y zona horaria.
4. Permitir buscar ciudades.
5. Crear páginas individuales para ciudades.
6. Mostrar información temporal relevante.
7. Permitir comparar varias ciudades.
8. Ser completamente responsive.
9. Tener una arquitectura SEO-first.
10. Tener una identidad visual consistente con 941.
11. Ser extremadamente rápida.
12. Estar preparada para monetización mediante AdSense.

4. No objetivos del MVP

No se deberá intentar construir inicialmente:

- aplicación móvil nativa
- sistema de cuentas complejo
- API pública
- sistema empresarial
- calendario colaborativo
- IA avanzada

- sistema de suscripciones
- widgets externos
- todas las funcionalidades de Time.is

Estas funcionalidades podrán incorporarse posteriormente.

5. Usuarios objetivo

Usuario casual

Quiere saber:

“¿Qué hora es?”

Debe obtener la respuesta inmediatamente.

Viajero

Quiere saber:

“¿Qué hora es en mi destino?”

Usuario internacional

Trabaja con personas de otros países.

Necesita comparar horarios.

Profesional

Necesita encontrar una hora adecuada para reuniones.

Desarrollador

Puede utilizar posteriormente herramientas como Unix Timestamp o la API.

Usuario recurrente

Guarda varias ciudades y consulta 941 diariamente.

6. Propuesta de valor

941

Precise time. Beautifully simple.

La propuesta combina:

Precisión

- ●

Velocidad

- ●

Simplicidad

- ●

Diseño

- ●

Cobertura mundial

- ●

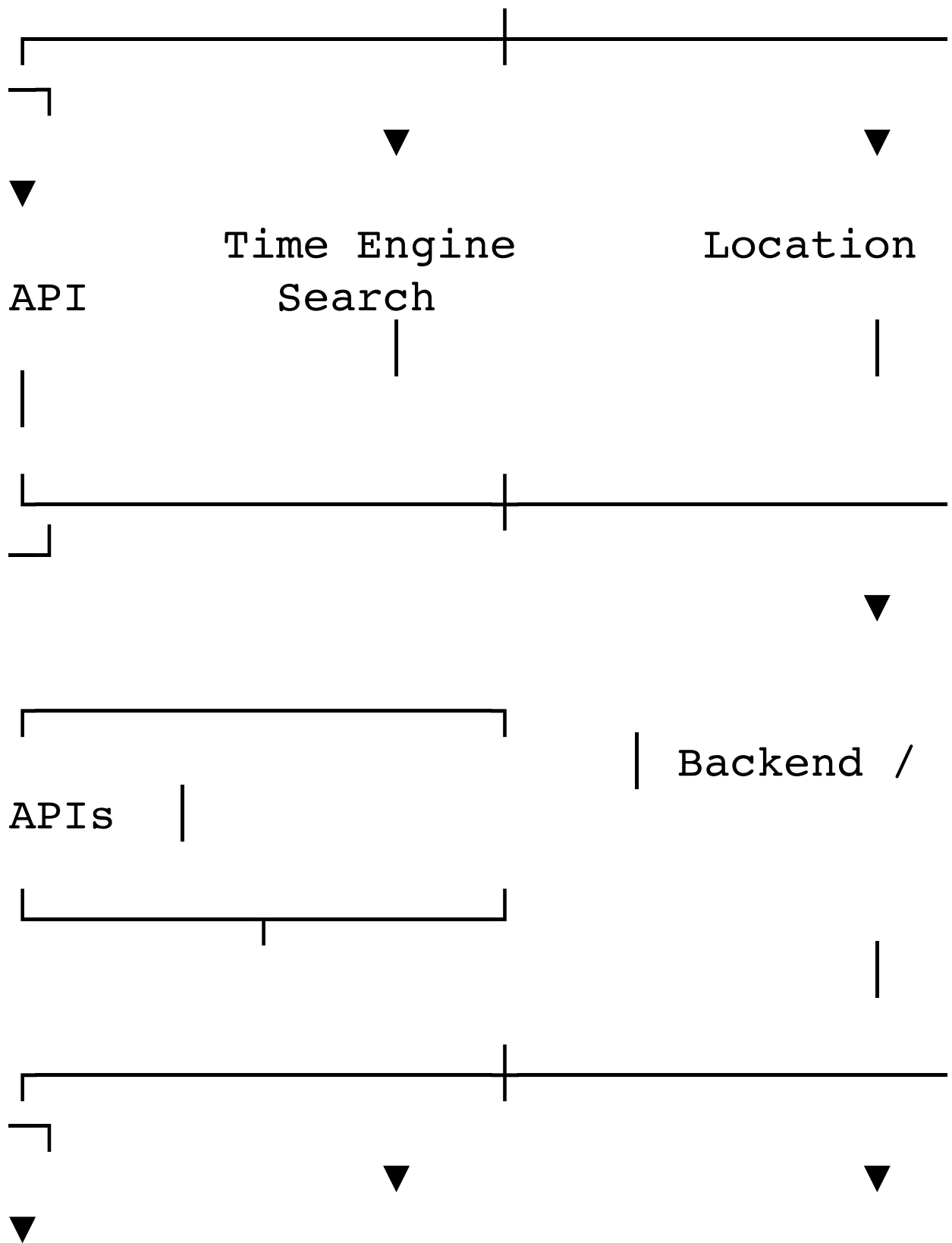
Herramientas

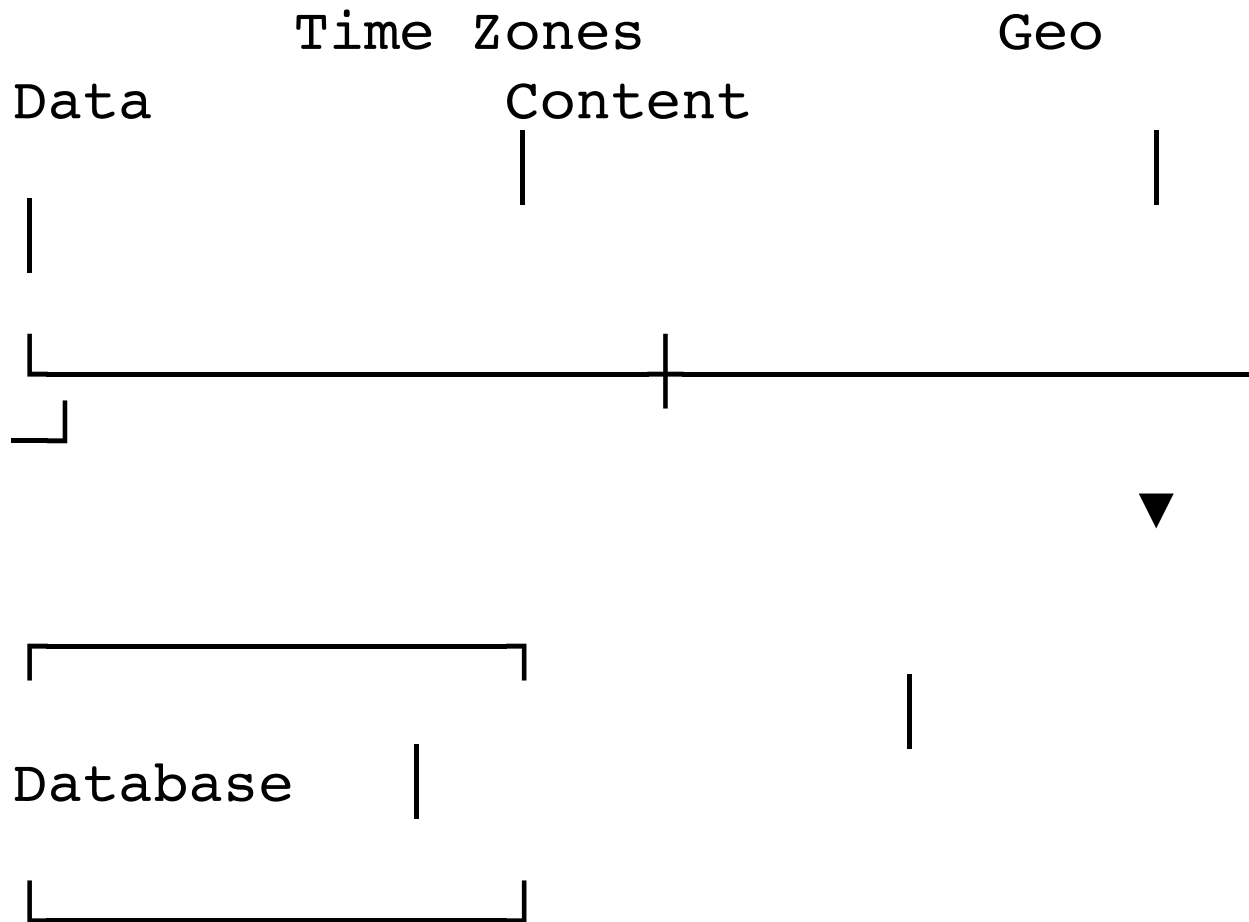
• ●
SEO

7. Arquitectura general

Conceptualmente:







8. Stack tecnológico

La implementación definitiva deberá ser validada por Antigravity, pero el proyecto deberá priorizar:

Frontend

Framework moderno orientado a rendimiento y SEO.

Preferencia:

Next.js + React + TypeScript

o una arquitectura equivalente que proporcione:

- SSR
- SSG
- routing
- metadata
- optimización
- excelente rendimiento

Backend

TypeScript / Node.js

o infraestructura equivalente.

Base de datos

Preferencia inicial:

PostgreSQL

Cache

Redis o mecanismo equivalente.

Hosting

Arquitectura cloud escalable.

Una posible implementación:

Frontend

→ **Vercel / Cloudflare**

Backend

→ Serverless / Containers

Database

→ PostgreSQL

Cache

→ Redis

CDN

→ Cloudflare

Esto no debe considerarse obligatorio todavía. Antigraity deberá evaluar la arquitectura final según requisitos de coste, rendimiento y escalabilidad.

9. Motor temporal

El motor temporal será uno de los componentes críticos.

Debe proporcionar:

- UTC
- timestamp
- offset
- zona horaria
- fecha

- hora
- DST
- conversión

El cliente no deberá depender únicamente del reloj del dispositivo.

10. Sincronización

El sistema deberá obtener una referencia temporal del servidor.

Conceptualmente:

Server timestamp



Client receives timestamp



Measure network latency



Calculate offset



Local high-resolution clock



Display corrected time

El reloj podrá actualizarse visualmente cada segundo, mientras que el motor interno podrá utilizar mayor resolución cuando sea necesario.

11. Precisión

El sistema deberá:

- minimizar drift
- compensar latencia
- sincronizar periódicamente
- manejar pérdida temporal de conexión
- detectar diferencias significativas entre servidor y cliente

No se deberá prometer públicamente una precisión determinada hasta haber realizado pruebas reales.

12. Time Zone Database

La plataforma deberá utilizar **IANA Time Zone Database** como referencia principal.

Cada ciudad deberá estar asociada a una zona válida.

Ejemplo:

Santo Domingo

→ `America/Santo_Domingo`

La aplicación deberá soportar actualizaciones de la base de zonas horarias.

13. Modelo de datos

Modelo conceptual de ciudad:

City

id

name

slug

country

countryCode

latitude

longitude

timezone

population

region

active

createdAt

updatedAt

Zona horaria:

TimeZone

id

ianaName

countryCodes

utcOffset

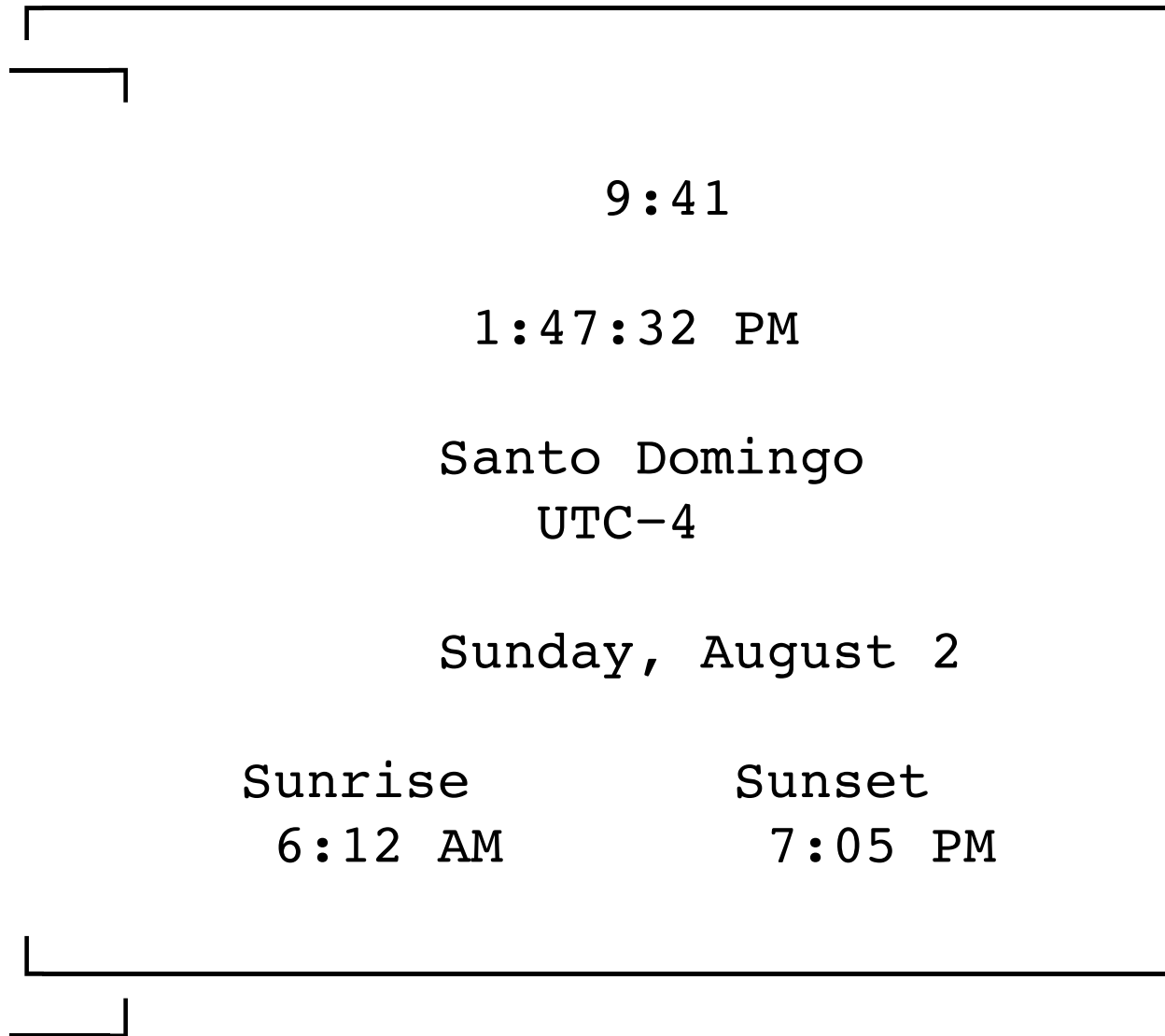
dst

displayName

14. Home

La página principal será extremadamente minimalista.

Conceptualmente:



La hora deberá ser el elemento visual dominante.

15. Header

El header deberá incluir:

Logo 941

y navegación mínima.

Propuesta:

941

Time

World Clock

Compare

Tools

En móvil:

941



16. Buscador

Debe ser uno de los elementos fundamentales.

Placeholder:

Search a city or country

Debe aceptar:

- ciudad
- país
- zona horaria
- código UTC

Ejemplos:

New York

Santo Domingo

Tokyo

Madrid

UTC-4

17. Autocomplete

Al escribir:

san

podrá aparecer:

Santo Domingo

San Francisco

San Juan

San Diego

Cada resultado deberá mostrar:

- ciudad
- país
- zona horaria

18. Página de ciudad

Ejemplo conceptual:

941

New York

2:47:31 PM

Sunday, August 2, 2026

Eastern Time

UTC-4

Sunrise

6:03 AM

Sunset

8:02 PM

Debajo:

- diferencia con hora local
- otras ciudades
- información relacionada
- enlaces internos

19. URL structure

La estructura deberá ser sencilla y legible.

Ejemplos:

```
/time/new-york  
/time/santo-domingo  
/time/tokyo  
/time/madrid
```

Alternativamente se podrá estudiar:

```
/new-york  
/santo-domingo
```

La decisión final deberá considerar SEO, escalabilidad y posibles conflictos de rutas.

20. SEO programático

Cada página indexable deberá generar automáticamente:

Title

Ejemplo:

Current Time in New York, USA | 941

Description

Descripción única y útil.

H1

Current Time in New York

Datos estructurados

Cuando corresponda.

Internal linking

Enlaces hacia:

- ciudades cercanas
- país
- zona horaria
- ciudades populares
- comparadores

21. Sitemap

El sistema deberá generar automáticamente:

`/sitemap.xml`

o sitemap index cuando la cantidad de URLs lo requiera.

Deberá poder soportar potencialmente cientos de miles o millones de URLs sin generar un archivo inmanejable.

22. Robots

Deberá existir:

`/robots.txt`

permitiendo a los motores de búsqueda rastrear las páginas públicas útiles.

Las páginas internas, parámetros innecesarios y endpoints deberán controlarse.

23. Canonical

Cada página indexable deberá tener canonical correctamente configurado.

Esto será particularmente importante para:

- búsquedas
- filtros
- parámetros
- traducciones
- páginas duplicadas

24. World Clock

El usuario podrá agregar ciudades.

Ejemplo:

WORLD CLOCK

Santo Domingo	1:47 PM
New York	1:47 PM
Los Angeles	10:47 AM
London	6:47 PM
Madrid	7:47 PM
Tokyo	2:47 AM

El reloj deberá actualizarse simultáneamente.

25. Comparador

El usuario seleccionará:

Ciudad A

Ciudad B

El sistema mostrará:

Santo Domingo
1:47 PM

New York
1:47 PM

Difference
0 hours

Para casos distintos:

Santo Domingo

1:47 PM

Madrid

7:47 PM

Difference

+6 hours

26. Visualización temporal

Una futura versión deberá incluir una línea temporal.

Ejemplo:

00 06 12 18 24
|-----|-----|-----|-----|

Tokyo



Madrid



New York



Santo Domingo



Esto permitirá comprender diferencias horarias visualmente.

27. Meeting Planner

Funcionalidad futura de alta prioridad.

Input:

Santo Domingo

New York

Madrid

Tokyo

Output:

BEST MEETING TIMES

09:00 AM — 10:00 AM

El sistema deberá considerar:

- horario laboral
- día
- zona horaria
- diferencia de fechas

28. Sunrise / Sunset

Cada ciudad podrá mostrar:

- sunrise
- sunset
- day length

La información deberá calcularse utilizando coordenadas geográficas y algoritmos o servicios confiables.

29. Countdown

Ruta:

`/countdown`

Debe permitir:

- seleccionar fecha
- seleccionar hora
- seleccionar timezone
- crear nombre del evento

Ejemplo:

`NEW YEAR`

`147 days`

`08 hours`

`21 minutes`

`32 seconds`

30. Timer

Ruta:

`/timer`

Funciones:

- Start

- Pause
- Resume
- Reset

La experiencia debe ser extremadamente simple.

31. Unix Timestamp

Ruta:

`/unix-time`

Conversión:

Unix timestamp

→

Human readable date

y:

Date

→

Unix timestamp

32. Calendario

Ruta:

`/calendar`

MVP:

- mes

- días
- fecha actual
- número de semana

Futuro:

- feriados
- calendarios regionales
- eventos

33. Favoritos

El usuario podrá guardar ciudades.

Para MVP se puede utilizar:

localStorage

sin necesidad de crear cuenta.

Posteriormente:

Account

→

Cloud Favorites

34. Cuenta de usuario

No será necesaria en MVP.

Si se incorpora posteriormente:

- email

- OAuth
- favoritos sincronizados
- preferencias
- configuración

La cuenta no deberá ser necesaria para utilizar las funciones básicas.

35. Preferencias

El usuario podrá configurar:

Hora

12 horas / 24 horas

Segundos

Mostrar / ocultar

Fecha

Formato regional

Primer día de semana

Domingo / lunes

36. Dark Mode

Aunque la identidad principal será azul cielo, azul oscuro y blanco, el sistema deberá contemplar una interfaz oscura futura.

No deberá introducir negro puro como color de marca.

La base podrá ser:

Deep Navy

en lugar de negro.

37. Sistema de diseño

Todos los componentes deberán utilizar tokens.

Ejemplo conceptual:

```
--color-sky  
--color-navy  
--color-white
```

```
--radius-sm  
--radius-md  
--radius-lg
```

```
--shadow-soft  
--shadow-card
```

`--font-display`

`--font-body`

Esto permitirá modificar el diseño desde un único lugar.

38. Logo

El logo oficial será el diseñado actualmente:

9:41 a.m.

con:

- fondo azul cielo
- números blancos
- sombra azulada
- esquinas redondeadas

Deberá existir en:

- SVG
- PNG
- favicon
- Apple Touch Icon
- Open Graph

39. Open Graph

Cada página deberá generar correctamente:

- título
- descripción

- imagen
- URL

Ejemplo:

Current Time in Santo Domingo | 941

La imagen podrá utilizar una composición dinámica de 941.

40. Rendimiento

Objetivo:

Lighthouse

Intentar alcanzar:

90+

en:

- Performance
- Accessibility
- Best Practices
- SEO

especialmente en páginas principales.

41. Core Web Vitals

Se deberán monitorizar:

- LCP
- INP
- CLS

La interfaz deberá minimizar:

- JavaScript
- layout shifts
- imágenes innecesarias
- fuentes pesadas

42. PWA

La arquitectura deberá estar preparada para convertirse en:

Progressive Web App.

Posteriormente podrá permitir:

Add 941 to Home Screen

sin necesidad de descargar inicialmente una aplicación nativa.

43. Monetización

MVP preparado para:

Google AdSense.

Los anuncios deberán:

- respetar la identidad visual
- no interferir con el reloj
- no aparecer de forma agresiva
- evitar afectar Core Web Vitals

La posición exacta deberá determinarse mediante pruebas.

44. Analítica

Inicialmente se podrá integrar una solución como:

Google Analytics 4

y:

Google Search Console

Se deberán medir eventos como:

`city_search`

`city_view`

`city_added`

`comparison_created`

`timer_started`

`countdown_created`

`unix_conversion`

45. Error handling

La aplicación deberá manejar:

- API offline
- ciudad no encontrada
- timezone inválida
- servidor no disponible
- timeout
- errores de búsqueda

Nunca mostrar errores técnicos incomprensibles al usuario.

46. Disponibilidad

El reloj principal debe tener mecanismos de fallback.

Si falla temporalmente una API externa:

la hora previamente sincronizada debe continuar funcionando localmente, dentro de los límites de precisión que pueda mantener el navegador.

47. Seguridad

Implementar:

- HTTPS
- headers de seguridad
- CSP cuando sea viable
- rate limiting
- validación

- sanitización
- protección de APIs
- secretos exclusivamente en variables de entorno

Nunca colocar API keys privadas en el frontend.

48. Privacidad

941 deberá recopilar solamente la información necesaria.

La detección de ubicación deberá utilizar preferentemente:

- zona horaria
- ubicación aproximada
- permisos explícitos cuando sea necesario

No deberá almacenarse una ubicación precisa sin una razón clara y consentimiento apropiado.

49. Escalabilidad

El sistema deberá estar preparado para:

10,000 usuarios

↓

100,000

↓

1,000,000

↓

10,000,000+

La arquitectura deberá minimizar operaciones dinámicas innecesarias.

Las páginas de ciudades podrán aprovechar:

Static Generation + CDN + cache.

Esto será fundamental para SEO y costes.

50. Internacionalización

La arquitectura deberá soportar múltiples idiomas.

Primera versión:

English

Segunda prioridad:

Spanish

Posteriormente:

- Portuguese
- French
- German
- Italian
- otros idiomas según tráfico.

51. Criterios de aceptación del MVP

El MVP será considerado funcional cuando:

Reloj

- muestra hora correcta
- muestra segundos
- sincroniza con servidor
- funciona en móvil
- continúa funcionando durante pérdida temporal de conexión

Ciudades

- búsqueda
- autocomplete
- página individual
- timezone correcta
- hora correcta

SEO

- metadata
- sitemap
- robots
- canonical
- URLs limpias
- SSR/SSG
- schema cuando corresponda

Diseño

- identidad 941

- azul cielo
- azul oscuro
- blanco
- sin negro en branding
- responsive
- accesible

Rendimiento

- Lighthouse 90+ objetivo
- Core Web Vitals saludables
- carga rápida

52. Fases de desarrollo

Fase 0 — Foundation

- repositorio
- arquitectura
- TypeScript
- framework
- diseño
- tokens
- CI/CD
- variables de entorno

Fase 1 — Core Clock

- time engine
- sincronización
- timezone

- reloj
- fecha
- ubicación

Fase 2 — Cities

- database
- búsqueda
- ciudades
- páginas dinámicas
- SEO

Fase 3 — World Clock

- múltiples ciudades
- comparación
- favoritos

Fase 4 — Tools

- countdown
- timer
- Unix
- calendar

Fase 5 — Advanced

- Meeting Planner
- timeline
- sunrise/sunset
- visualizaciones

Fase 6 — Growth

- AdSense
- analytics
- SEO avanzado
- internacionalización

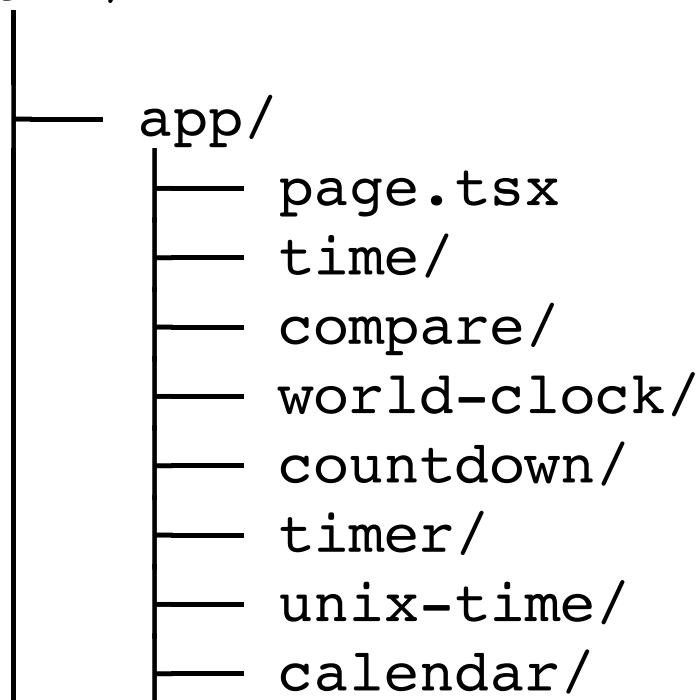
Fase 7 — Platform

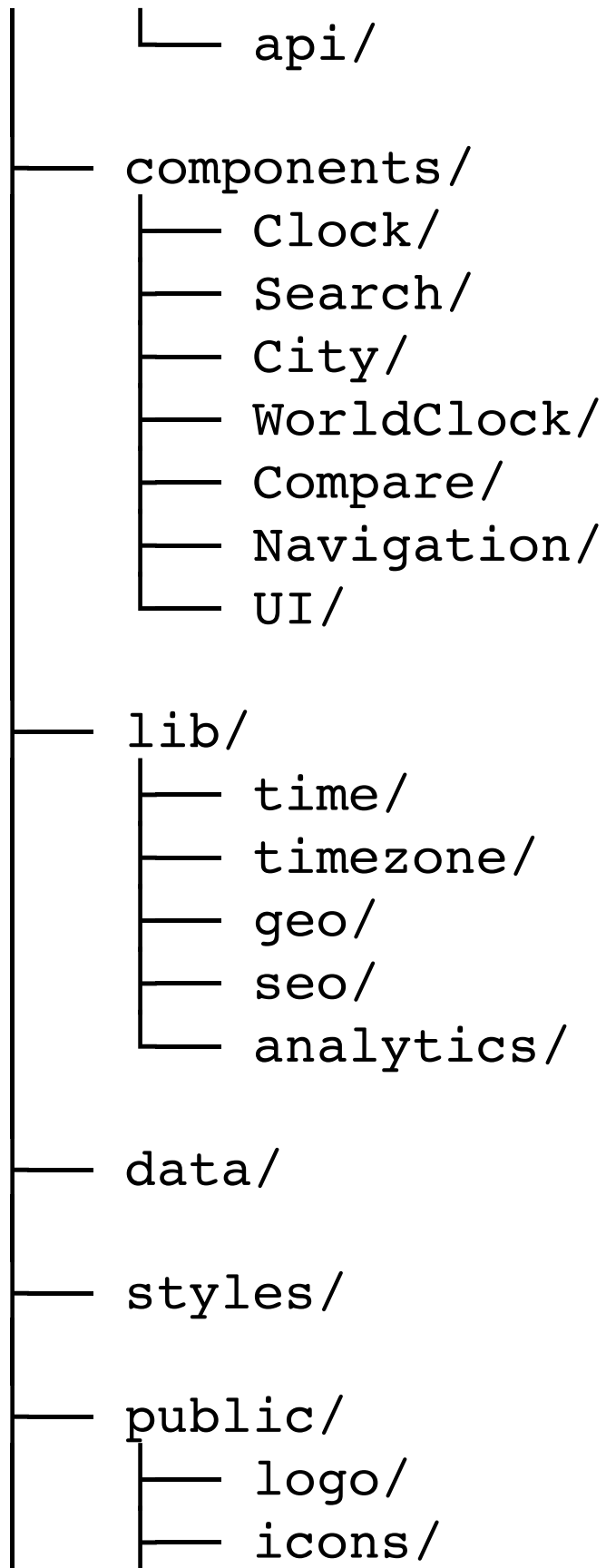
- PWA
- API
- aplicaciones
- Premium

53. Arquitectura de carpetas propuesta

Si utilizamos Next.js:

941 /






```
└─ images/  
└─ tests/
```

54. Testing

Deberán existir pruebas para:

Unit tests

- conversión horaria
- timezone
- DST
- timestamps
- fechas

Integration

- búsqueda
- API
- páginas de ciudades

E2E

- navegación
- búsqueda
- comparación
- timer
- responsive

Las zonas horarias deberán recibir especial atención porque los errores de DST pueden generar resultados incorrectos.

55. Principio de desarrollo para Antigravity

Antigravity deberá trabajar de forma incremental.

No generar todo el proyecto de una sola vez.

Orden recomendado:

Foundation

↓

Design System

↓

Time Engine

↓

Home

↓

Cities

↓

SEO

↓

World Clock

↓

Tools



Analytics



Monetization

Después de cada fase:

test → revisar → corregir → continuar.

56. Regla fundamental para Antigravity

Antigravity deberá evitar:

- sobreingeniería
- dependencias innecesarias
- componentes duplicados
- código difícil de mantener
- diseños genéricos
- colores fuera de la identidad
- interfaces saturadas
- copiar directamente a [Time.is](https://time.is)

Y deberá priorizar:

performance + simplicidad + mantenibilidad + SEO + UX.

57. Criterio de diseño definitivo

Si existe un conflicto entre:

más funcionalidades

y

mejor experiencia

941 deberá elegir mejor experiencia.

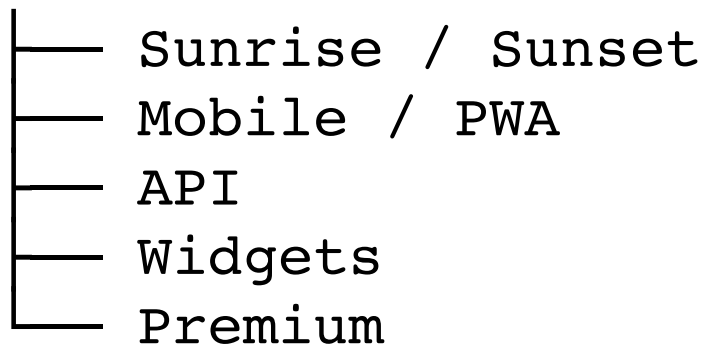
El producto nunca debe convertirse en un dashboard.

58. Visión de largo plazo

La evolución prevista:

941

- Current Time
- World Clock
- Time Zones
- Compare
- Meeting Planner
- Calendar
- Countdown
- Timer
- Unix Time



Todo debe permanecer unido bajo una misma filosofía:

Time, beautifully simple.

59. Definición final del producto

941 no es un reloj.

Es una **plataforma global de información temporal.**

El reloj es la puerta de entrada.

Las ciudades son el contenido.

Las zonas horarias son la infraestructura.

Las herramientas son el ecosistema.

El SEO es el motor de adquisición.

La simplicidad es la filosofía.

Y **9:41** es la identidad.

60. Documento maestro para Antigravity

Este PRD debe considerarse la **especificación funcional inicial de 941 v1.0**.

Pero hay una distinción importante:

Todavía no le daría este documento completo a Antigravity para que empiece a programar.

Antes haría un documento adicional:

941 — Technical Architecture Document (TAD)

Ese documento bajaría el PRD a decisiones concretas sobre:

- arquitectura exacta
- APIs
- proveedores de datos
- base de datos
- IANA
- geocoding
- sunrise/sunset
- sincronización NTP
- caché
- CDN
- estructura exacta de URLs
- estrategia de millones de páginas SEO
- schema.org
- AdSense
- analytics

- seguridad
- CI/CD
- costos estimados
- variables de entorno
- endpoints
- modelos SQL
- componentes React
- y, sobre todo, **el prompt maestro que entregaremos a Google Antigravity.**

Ese sería el siguiente documento antes de escribir la primera línea de código.